



Tarefa 1

O Início da Operação

MC322 - Programação Orientada a Objetos

Professor: Marcelo da Silva Reis

PEDs: Athyrson Ribeiro, Matheus Lozano

PADs: João Pedro Frare

1. Descrição Geral

Uma planta industrial automatizada é composta por diversos equipamentos responsáveis por transformar matérias-primas em produtos acabados. Para que a produção ocorra corretamente, é necessário monitorar o estado dos equipamentos, transportar materiais entre diferentes etapas e executar operações específicas em cada estação da planta.

Ao longo da disciplina, desenvolveremos um sistema de *software* capaz de representar e simular uma planta industrial automatizada. O projeto será construído em etapas e as tarefas serão **incrementais**, de forma que cada tarefa utilize e estenda o que foi desenvolvido anteriormente (ou seja, a Tarefa 2 será desenvolvida em cima do que foi desenvolvido na Tarefa 1, a 3 na 2, e assim por diante).

Nesta primeira tarefa, vocês darão os primeiros passos na construção da planta. O objetivo é implementar uma versão simplificada de uma linha de produção. Neste momento, a planta será bem simples: ela será composta por matérias-primas, produtos, equipamentos e uma linha de produção. Cada dupla terá a liberdade de fazer escolhas sobre a estrutura do projeto e a temática da fábrica (roupas, peças automotivas, componentes eletrônicos, alimentos etc.), dentro dos limites da especificação de cada laboratório.

1.1. Entendendo a Planta Industrial

A planta industrial opera da seguinte forma:

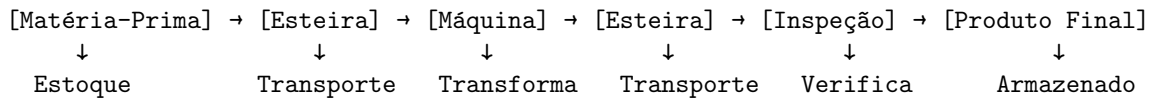
1. **Entrada de Matéria-Prima:** A matéria-prima chega à planta e é armazenada em estoque. Cada matéria-prima possui uma quantidade disponível que será consumida durante o processo de fabricação.
2. **Transporte:** A matéria-prima é colocada em uma esteira que a transporta até a máquina de processamento. A esteira deve estar em movimento para realizar o transporte.
3. **Processamento:** A máquina recebe a matéria-prima e a transforma em produto acabado. Para isso, a máquina deve estar ligada e a quantidade de matéria-prima deve ser suficiente para atender à demanda do produto.
4. **Transporte do Produto:** O produto finalizado é colocado novamente na esteira e transportado até a estação de inspeção.
5. **Inspeção:** A estação de inspeção verifica a qualidade do produto. Nesta primeira versão, a inspeção é simplificada, apenas marcando o produto como aprovado.
6. **Produto Final:** Após a inspeção, o produto está pronto para ser armazenado ou despachado.

Cada equipamento possui estados próprios e pode executar ações sobre os materiais. Por exemplo:

- Uma máquina pode estar **desligada** ou **ligada**, e só processa quando ligada;
- Uma esteira pode estar **parada** ou **em movimento**, e só transporta quando em movimento;
- Uma estação de inspeção pode estar **desativada** ou **ativa**, e só inspeciona quando ativa;
- Uma matéria-prima pode estar **disponível** ou **esgotada**, dependendo da quantidade em estoque;
- Um produto pode estar **aguardando processamento**, **processado** ou **inspecionado**.

1.2. Fluxo de Produção

O fluxo completo de produção segue a seguinte sequência:



1.3. Características Importantes

- **Demanda de Matéria-Prima:** Cada produto possui uma demanda específica de matéria-prima que deve ser atendida para sua fabricação. O sistema deve verificar se há estoque suficiente antes de iniciar a produção.
- **Controle de Estoque:** A quantidade de matéria-prima em estoque deve ser atualizada a cada produção, refletindo o consumo realizado.
- **Estados dos Equipamentos:** Os equipamentos devem respeitar seus estados. Uma máquina desligada não processa, uma esteira parada não transporta, e uma estação de inspeção desativada não inspeciona.
- **Identificação:** Cada matéria-prima e produto deve possuir um identificador único para rastreabilidade.
- **Entrada de Dados:** O sistema deve aceitar apenas entradas numéricas do usuário. Nomes de produtos, matérias-primas e equipamentos são definidos diretamente no código-fonte.

A planta industrial simplificada serve como base para o desenvolvimento incremental do sistema. Nas próximas tarefas, novos equipamentos, funcionalidades e complexidades serão adicionados a esta estrutura inicial.

2. Objetivos

Ao final desta tarefa, o aluno deverá ser capaz de:

- Familiarizar-se com a linguagem Java (versão 25) e a IDE escolhida (VSCode);
- Compreender os conceitos fundamentais de Programação Orientada a Objetos: classes, atributos e métodos;
- Implementar classes concretas em Java com atributos e métodos bem definidos;
- Modelar entidades básicas de uma planta industrial utilizando orientação a objetos;
- Implementar encapsulamento por meio de atributos privados e métodos públicos;
- Organizar o código em múltiplas classes de forma coesa;
- Criar um programa simples de interação via terminal;
- Instanciar e manipular objetos de diferentes classes;
- Reutilizar objetos por meio da composição entre classes.

Observação: Nesta primeira tarefa, não será utilizada herança. As classes deverão ser implementadas de forma independente, mesmo que alguns equipamentos compartilhem características semelhantes. A introdução de herança e classes abstratas será realizada na Tarefa 2.

3. Atividades

Esta seção descreve a implementação principal da tarefa. As classes a seguir representam a estrutura mínima esperada, mas sintam-se encorajados a expandi-la. As seguintes diretrizes gerais devem ser observadas:

- **Liberdade Criativa:** A planta apresentada neste enunciado é apenas um exemplo. Você pode personalizar nomes, produtos e características dos equipamentos, desde que a estrutura mínima solicitada seja mantida.

- **Autonomia Técnica:** Este enunciado é intencionalmente vago em alguns aspectos, pois parte da atividade consiste na tomada de decisões de *design*. Espera-se que você:
 - Identifique os tipos de variáveis mais adequados para cada atributo;
 - Determine quais parâmetros são necessários para cada método;
 - Escolha valores iniciais coerentes para os atributos.

O projeto deverá representar uma **linha de produção simples capaz de receber matéria-prima, processá-la e transformá-la em produto final**, considerando a **demanda variável de matéria-prima** para cada tipo de produto.

3.1. Implementação das Classes

Devem ser implementadas as seguintes classes concretas:

3.1.1. Classe MateriaPrima

Representa a matéria-prima que será transformada em produto pela planta industrial.

Atributos

- **id** – Identificador único da matéria-prima;
- **nome** – Nome ou tipo da matéria-prima (definido no código, não pelo usuário);
- **quantidade** – Quantidade disponível em estoque;
- **unidade** – Unidade de medida (definida no código);
- **quantidadeMinima** – Quantidade mínima aceitável para produção.

Métodos

- **consumir** – Reduz a quantidade disponível da matéria-prima, recebendo como parâmetro a quantidade demandada;
- **adicionarEstoque** – Adiciona quantidade ao estoque de matéria-prima;
- **verificarDisponibilidade** – Verifica se há quantidade suficiente para atender uma demanda específica;
- **getId** – Retorna o identificador da matéria-prima;
- **getNome** – Retorna o nome da matéria-prima;
- **getQuantidade** – Retorna a quantidade disponível.

3.1.2. Classe Produto

Representa o produto final que será fabricado a partir da obra-prima.

Atributos

- **id** – Identificador único do produto;
- **nome** – Nome ou tipo do produto (definido no código);
- **status** – Estado atual do produto na linha de produção;
- **quantidadeMateriaPrimaNecessaria** – Quantidade de matéria-prima necessária para produzir uma unidade deste produto.

Métodos

- **processar** – Marca o produto como processado;
- **definirDemandaMateriaPrima** – Define a quantidade de matéria-prima necessária para produzir o produto;
- **getDemandaMateriaPrima** – Retorna a quantidade de matéria-prima necessária;
- **getId** – Retorna o identificador do produto;
- **getNome** – Retorna o nome do produto;
- **getStatus** – Retorna o status atual do produto.

3.1.3. Classe Maquina

Representa uma máquina utilizada para transformar matéria-prima em produto durante o processo produtivo.

Atributos

- **nome** – Nome da máquina (definido no código);
- **ligada** – Indica se a máquina está ligada;
- **capacidadeMaxima** – Capacidade máxima de processamento por ciclo.

Métodos

- **ligar** – Liga a máquina;
- **desligar** – Desliga a máquina;
- **processar** – Transforma a matéria-prima em produto, recebendo como parâmetros a obra-prima e a demanda necessária;
- **getNome** – Retorna o nome da máquina;
- **estaLigada** – Retorna se a máquina está ligada.

Uma máquina desligada não deve ser capaz de processar produtos.

3.1.4. Classe Esteira

Representa uma esteira utilizada para transportar itens entre as etapas da planta.

Atributos

- **item** – Item atualmente transportado pela esteira (pode ser matéria-prima ou produto);
- **emMovimento** – Indica se a esteira está em funcionamento;
- **capacidadeMaxima** – Capacidade máxima de peso ou volume que a esteira pode transportar.

Métodos

- **ligar** – Coloca a esteira em funcionamento;
- **desligar** – Interrompe o funcionamento da esteira;
- **adicionarItem** – Coloca um item na esteira;
- **removerItem** – Remove e retorna o item da esteira;
- **verificarCapacidade** – Verifica se a esteira suporta o peso/volume do item.

Uma esteira que já contém um item não deve aceitar outro.

3.1.5. Classe EstacaoInspecao

Representa uma estação responsável por realizar uma inspeção simples nos produtos fabricados.

Atributos

- **ativa** – Indica se a estação está ativa;
- **produtosInspeccionados** – Contador de produtos inspeccionados.

Métodos

- **ativar** – Ativa a estação;
- **desativar** – Desativa a estação;
- **inspecionar** – Realiza a inspeção de um produto;
- **getTotalInspeccionados** – Retorna o total de produtos inspeccionados.

Uma estação de inspeção desativada não deve realizar inspeções.

3.2. Sistema da Planta via Terminal

Para testar a implementação das classes, deverá ser desenvolvido um programa principal, por exemplo na classe **Main**. O programa deverá:

1. Instanciar pelo menos uma matéria-prima com estoque inicial;
2. Instanciar produtos pré-definidos no código;
3. Instanciar uma máquina de processamento;
4. Instanciar uma esteira;
5. Instanciar uma estação de inspeção;
6. Exibir um menu com opções numéricas;
7. Permitir que o usuário selecione qual produto deseja produzir (via número);
8. Permitir que o usuário defina a demanda de matéria-prima (quantidade numérica);
9. Verificar se há matéria-prima suficiente para atender à demanda;
10. Ligar os equipamentos necessários;
11. Colocar a matéria-prima na esteira;
12. Transportar a matéria-prima até a máquina;
13. Processar a matéria-prima na máquina, considerando a demanda definida;
14. Transportar o produto pela esteira até a inspeção;
15. Realizar a inspeção do produto;
16. Permitir que o usuário consulte o estoque de matéria-prima;
17. Informar ao usuário o estado da produção ao longo da execução.

A interação deve ser realizada utilizando **Scanner** para leitura de entrada. **Importante:** O programa deve aceitar **apenas entradas numéricas** do usuário. Nomes de produtos, matérias-primas e máquinas devem ser definidos diretamente no código-fonte. Um exemplo simplificado de execução seria:

```

=====
PLANTA INDUSTRIAL
=====

Matéria-Prima: MP001 - Plástico
Quantidade: 100 kg
Unidade: kg

Produtos Disponíveis:
1 - Componente Plástico (demanda: 15 kg)
2 - Tampa Plástica (demanda: 5 kg)
3 - Suporte Plástico (demanda: 25 kg)

=====
MENU PRINCIPAL
=====
1 - Iniciar produção
2 - Consultar estoque
3 - Sair

Escolha: 1

Selecione o produto (1-3): 1

Informe a demanda de matéria-prima (kg): 15

[OK] Verificando disponibilidade de Plástico...
[OK] Demanda de 15 kg pode ser atendida.

[OK] Esteira ligada.
[OK] Máquina ligada.
[OK] Matéria-prima MP001 colocada na esteira.
[OK] Matéria-prima transportada até a máquina.
[OK] Máquina processando 15 kg de Plástico...
[OK] Produto P001 - Componente Plástico criado.
[OK] Produto P001 transportado para inspeção.
[OK] Estação de inspeção ativada.
[OK] Produto P001 aprovado na inspeção.

=====
PRODUÇÃO CONCLUÍDA COM SUCESSO
=====

Estoque restante de Plástico: 85 kg

```

O exemplo acima é apenas ilustrativo. A organização das mensagens, opções disponíveis e fluxo de interação ficam a critério do aluno, desde que os comportamentos mínimos especificados sejam implementados e que **apenas entradas numéricas sejam aceitas**.

3.3. Pontos de Atenção

Teste sua aplicação e verifique se todos os componentes da planta funcionam corretamente. Preste atenção especialmente aos seguintes pontos:

- Os atributos das classes devem ser privados;
- O acesso aos atributos deve ocorrer por meio de métodos públicos apropriados;
- Uma máquina desligada não deve processar produtos;
- Uma esteira desligada não deve transportar itens;
- Uma esteira que já possui um item não deve aceitar outro;

- Uma estação de inspeção desativada não deve realizar inspeções;
- A demanda de matéria-prima deve ser verificada antes do processamento;
- A quantidade de matéria-prima consumida deve corresponder à demanda definida;
- O estoque de matéria-prima deve ser atualizado após cada produção;
- O usuário deve ser informado quando não houver matéria-prima suficiente;
- Apenas entradas numéricas devem ser aceitas do usuário;
- Nomes de produtos, matérias-primas e máquinas devem estar definidos no código;
- O fluxo de produção deve ser coerente com os estados dos equipamentos;
- Cada classe deve ser responsável por seus próprios dados e comportamentos.

4. Avaliação

A nota do laboratório será composta pelos seguintes critérios:

1. Modelagem e Implementação das Classes (2,0 pontos)

- Implementação correta das classes solicitadas (`MateriaPrima`, `Produto`, `Maquina`, `Esteira`, `EstacaoInspecao`);
- Presença dos atributos e métodos mínimos especificados;
- Uso adequado de encapsulamento (atributos privados e métodos de acesso);
- Implementação correta da relação entre matéria-prima e produto;
- Definição adequada da demanda de matéria-prima para cada produto.

2. Funcionamento da Planta e Interação via Terminal (2,0 pontos)

- Funcionamento correto da sequência de produção (matéria-prima → processamento → produto);
- Transporte adequado de materiais pela esteira;
- Processamento correto da matéria-prima pela máquina;
- Verificação e atualização do estoque de matéria-prima;
- Inspeção adequada dos produtos;
- Tratamento adequado dos estados dos equipamentos;
- Validação de entradas numéricas do usuário;
- Interação funcional via terminal.

3. Organização de Qualidade do Código (2,0 pontos)

- Cada classe está em seu próprio arquivo `.java`;
- Organização adequada dos arquivos;
- Código legível e bem formatado;
- Nomes adequados para classes, atributos e métodos;
- Comentários quando necessários para trechos não triviais;
- Separação adequada de responsabilidades entre as classes.

4. Funcionamento Completo da Simulação (1,0 pontos)

- A planta funciona do início ao fim sem interrupções inesperadas;
- O fluxo geral do programa é coerente e consistente;
- O controle de estoque de matéria-prima funciona corretamente;
- A demanda de matéria-prima é verificada antes do processamento;
- O sistema informa adequadamente quando não há matéria-prima suficiente;
- O usuário pode consultar o estoque e adicionar matéria-prima quando necessário.

5. Criatividade e Personalização (3,0 pontos)

A criatividade será avaliada de forma mais ampla nesta tarefa, considerando não apenas as funcionalidades implementadas, mas também a originalidade e o cuidado na concepção do projeto. Os seguintes aspectos serão considerados:

Escolha do Tema da Fábrica

Os alunos **devem** escolher um tipo de fábrica que seja interessante e significativo para a dupla. Alguns exemplos de temas possíveis:

- Fábrica de instrumentos musicais;
- Fábrica de bicicletas;
- Fábrica de brinquedos de madeira;
- Fábrica de produtos de papelaria;

Os exemplos acima são apenas sugestões. A dupla tem total liberdade para escolher qualquer tipo de fábrica que desperte seu interesse, desde que seja possível representar o fluxo de transformação de matéria-prima em produto.

Tela de Introdução

Ao iniciar o programa, deve ser exibida uma **tela de introdução** que apresente brevemente a fábrica escolhida. Esta introdução deve incluir:

- Nome da fábrica;
- Tipo de produto fabricado;
- Matéria-prima principal utilizada;
- Identificação da dupla (nomes dos integrantes).

Exemplo de tela de introdução:

```
=====
FÁBRICA DE SUCO NATURAL
"Da fruta direto para o seu copo"
=====
```

```
Bem-vindos à nossa fábrica de sucos!
Aqui transformamos frutas frescas em
sucos naturais e saborosos.
```

```
Desenvolvido por: Marina Rocha e Carlos Souza
=====
```

Arquivo de Justificativa (obrigatório)

A dupla deve criar um arquivo de texto chamado `justificativa.txt` contendo:

- **Justificativa da escolha do tema:** Explicação do porquê a dupla escolheu este tipo específico de fábrica (interesse pessoal, familiaridade com o tema, relevância social, potencial de expansão, etc.);
- **Visão de expansão futura:** Descrição de como a dupla imagina que a planta industrial poderá ser expandida nos próximos laboratórios (novos equipamentos, novas etapas de produção, novos produtos, automação adicional, etc.).

O arquivo deve ser escrito de forma clara e demonstrar reflexão genuína sobre o projeto, não apenas cumprir um requisito formal.

Toque Humano e Originalidade

Serão valorizadas implementações que demonstrem:

- **Personalidade no código:** Mensagens personalizadas, nomes criativos para equipamentos, referências ao tema escolhido;

- **Coerência temática:** Todos os elementos do programa (nomes, mensagens, fluxo) mantêm coerência com o tema da fábrica escolhido;
- **Detalhes cuidadosos:** Pequenos toques que demonstrem atenção e cuidado, como mensagens de erro contextualizadas, feedback visual agradável, ou *easter eggs* sutis;
- **Autenticidade:** Soluções que reflitam o estilo e a forma de pensar da dupla, evitando padronizações excessivas ou respostas genéricas;
- **Documentação pessoal:** Comentários no código que expliquem decisões de design com a "voz" da dupla, não apenas descrições técnicas impessoais.

Importante: Projetos que apresentem características de terem sido gerados de forma automática ou padronizada (como respostas típicas de IA) receberão pontuação reduzida neste critério. A avaliação buscará identificar o esforço genuíno e a personalidade da dupla no desenvolvimento.

Nota: As extensões funcionais são bem-vindas, mas a criatividade e personalização do projeto terão peso significativo na avaliação deste critério.

6. Penalidade por Não Compilação

Caso o projeto:

- Não compile utilizando Java 25; ou
- Não execute corretamente no ambiente de correção;

será aplicada uma penalidade de **-5,0 pontos** sobre a nota obtida.

A nota final mínima é **0,0** (não haverá notas negativas).

5. Entrega

A submissão é realizada em duas etapas: **repositório no GitHub** e **formulário do Google Forms**. Para isso:

1. Repositório no GitHub:

- O projeto deve estar hospedado em um repositório no GitHub;
- Gere uma **release (tag)** com a identificação do laboratório, seguindo o formato: **tarefaX-RA1-RA2**;
- A release deve conter todos os arquivos necessários para compilação e execução do projeto, incluindo o arquivo **justificativa.txt**.

2. Formulário do Google Forms:

- Um formulário será disponibilizado juntamente com a tarefa no Google Classroom;
- No formulário, a dupla deverá informar:
 - Nome completo e RA de cada integrante;
 - Link da release gerada;

Caso optem por manter o repositório **privado**, adicionem os PEDs da disciplina como colaboradores:

- Athyrson Ribeiro – @athyrson06
- Matheus Lozano – @lozavival

Lembrem-se que:

- O prazo de submissão é até o dia **02/09/2026 às 23h59**.
- Entregas atrasadas serão penalizadas em **25 % da nota obtida**. Não serão aceitas entregas atrasadas após 09/09/2026.
- Os horários de laboratório e atendimento podem (e devem) ser utilizados para tirar dúvidas relacionadas tanto à submissão quanto ao desenvolvimento do laboratório.

5.1. Compilação e Execução

Durante a avaliação, os PEDs utilizarão os seguintes comandos, a partir da raiz do repositório, para compilar e executar o projeto:

```
javac -d bin $(find src -name "*.java$")
java -cp bin Main
```

A classe principal do projeto (**Main** ou equivalente) deve conter o método:

```
public static void main(String[] args)
```

Dessa forma, a compilação e a execução ocorrerão corretamente no ambiente de avaliação.

6. Dicas de Boas Práticas

Um código de qualidade não é apenas aquele que funciona corretamente, mas também aquele que é **fácil de entender, manter e modificar**. Ao longo do desenvolvimento, é importante escrever código pensando não apenas na solução imediata, mas também na sua evolução futura.

Existem diversos princípios de construção de software que ajudam a atingir esses objetivos. A seguir, destacamos alguns dos mais relevantes:

- **DRY (Don't Repeat Yourself):** Evite duplicação de código. Se uma mesma lógica aparece em múltiplos lugares, considere abstrai-la em um método ou classe reutilizável. Por exemplo, a validação de entradas numéricas pode ser centralizada em métodos auxiliares.
- **KISS (Keep It Simple, Stupid):** Prefira soluções simples e diretas. Evite complexidade desnecessária, especialmente quando uma solução mais simples resolve o problema de forma adequada. E não se engane, isso não é ser preguiçoso. Muitas vezes chegar a uma solução simples exige mais esforço intelectual.
- **YAGNI (You Aren't Gonna Need It):** Evite implementar funcionalidades que não são necessárias no momento. Focar apenas no que é necessário ajuda a manter o código mais limpo e compreensível. Nesta primeira tarefa, não é necessário implementar funcionalidades avançadas como persistência de dados ou interface gráfica.

Outro conjunto importante de diretrizes são os princípios **SOLID**, que tratam de organização, responsabilidade e acoplamento entre classes. Embora tais princípios não sejam exigidos nas tarefas, sua compreensão pode ajudar a escrever código mais modular e flexível, facilitando para vocês mesmos darem seguimento nas próximas tarefas e no projeto final.

6.1. Considerações Específicas para Esta Tarefa

Além dos critérios gerais de avaliação, preste atenção especial aos seguintes aspectos:

- **Rastreabilidade:** O sistema deve permitir identificar qual matéria-prima foi utilizada na fabricação de cada produto.
- **Controle de Estoque:** A quantidade de matéria-prima deve ser atualizada corretamente após cada produção, refletindo o consumo realizado.
- **Validação de Demanda:** O sistema deve verificar se há matéria-prima suficiente antes de iniciar o processamento, informando o usuário caso a demanda não possa ser atendida.
- **Entradas Numéricas:** O programa deve tratar adequadamente entradas inválidas, solicitando que o usuário digite apenas números quando necessário.
- **Estados dos Equipamentos:** Cada equipamento deve respeitar seus estados. Uma máquina desligada não processa, uma esteira parada não transporta, e uma estação de inspeção desativada não inspeciona.
- **Capacidade dos Equipamentos:** A esteira e a máquina devem verificar se a demanda de matéria-prima não excede suas capacidades máximas.

Lembre-se: o objetivo desta tarefa é construir uma base sólida para as próximas etapas do projeto. Um código bem estruturado agora facilitará significativamente as expansões futuras da planta industrial.